

LLD Notes — Phase 3, Part 3

State pattern · Parking Lot (design round) | Concepts, your doubts, your mistakes, interview answers

Colour key. Blue = a doubt you asked · Red = a mistake you made · Green = rule to remember · Amber = how to say it in an interview.

Contents

- 10. State — the problem · structure · vending machine end to end · the context's three jobs · terminal states · when NOT to use it
- 11. Exercise 22 (support ticket) — the transition table · your Part B mistakes · the corrected code
- 12. Part C — scheduled jobs vs listeners · the audit choke point · the Expression Problem · concurrency and optimistic locking · defending the design
- 13. Parking Lot — the ten requirements · your Q1 answers · the four corrections · Q2–Q6 · the corrected blueprint
- 14. Updated mistake summary · requirement → pattern cheat sheet · interview scripts · recall questions

10. The State pattern

The problem it solves

An object behaves differently depending on what "mode" it is in, and the same operation is legal in some modes and illegal in others. Written naively, every operation grows a chain of `if` s:

```
public void resolve(String agent) {
    if (status == OPEN)          throw new IllegalStateException("assign it first");
    else if (status == ASSIGNED) { /* the real logic */ }
    else if (status == RESOLVED) throw new IllegalStateException("already resolved");
    else if (status == CLOSED)   throw new IllegalStateException("closed");
}
// ...and the same four-branch chain repeated in assign(), close(), reopen()
```

Four operations × four states = **16 branches** spread across four methods.

The trigger to recognise: you are writing the **same set of branches** in more than one method, and the branches are the object's *status*. That is the State pattern's signature.

The structure — three parts

Part	In the ticket example	Its job
State interface	TicketState	declares every operation the object supports
Concrete states	OpenState, AssignedState, ResolvedState, ClosedState	one class per state; each answers all operations
Context	SupportTicket	holds the data and the current state; delegates everything

The reorganisation is the point. The `if/else` version groups code **by operation** ("here is everything resolve does"). The State version groups code **by state** ("here is everything CLOSED allows").

I got this wrong first and you corrected me. I claimed you must "scan all functions" to answer "*can you reopen a closed ticket?*" You pushed back: the ticket already knows its state, and you'd just call `reopen()` . **You were right.** Neither layout is universally more readable — they are readable along different axes:
if/else answers "what does reopen do?" easily. **State** answers "what can CLOSED do?" easily.
Readability is *not* the real argument for State. The real argument is in section 12 (the compiler).

The context has exactly three jobs

```
class SupportTicket {

    private TicketState state = OpenState.INSTANCE; // 1. hold the current state
    private String agent;                          //    ...and all the data

    public void assign(String agent) { state.assign(this, agent); } // 2. delegate, one line
    public void resolve(String actor) { state.resolve(this, actor); }
    public void close() { state.close(this); }
    public void reopen() { state.reopen(this); }

    void setState(TicketState next) { this.state = next; } // 3. package-private hooks
    void setAgent(String agent) { this.agent = agent; }
    String getAgent() { return agent; }

}
```

Three rules for the context.
1. No if and no switch on the state, anywhere. If one appears, the pattern has failed.
2. Public methods are pure delegation — one line each, no logic.
3. The hooks are package-private, not public. Outside code must not be able to shove the object into any state it likes. (In section 12 this decision pays off: it makes `setState` the one choke point every transition must pass through.)

Why each state is a single shared instance

```
final class IdleState implements VendingMachineState {  
    static final IdleState INSTANCE = new IdleState();  
    private IdleState() { }  
    ...  
}
```

The state objects hold **no data** — every field lives on the context, which is why the context is passed into every method. A stateless object can be shared by every machine, every ticket, every thread, with no locking. One instance each, created once, used forever.

This is the Flyweight idea in miniature, and it is also why two vending machines can share the same four state objects while keeping completely independent balances and stock.

Vending machine — the worked example you asked for three times

Your doubts, answered together

"Do we need to know the flow of states before coding?" Yes — absolutely. **Draw the transition table first.** You cannot write the classes until you can fill every cell. That table is the design; the code is transcription.

"Do we always need a context holder class?" Yes. Something must own the data and the current state. Without it, the state objects would need data, and then they couldn't be shared.

"If the context is shared, do we make it a Singleton?" **No.** One vending machine = one context object. Ten machines = ten contexts. The *states* are shared singletons; the *context* is one per real-world thing. Confusing these is the most common State mistake.

"new VendingMachine() — at this point we're in IDLE?" Yes. The field initialiser `private VendingMachineState state = IdleState.INSTANCE;` runs during construction.

The transition table — write this before any code

	insertCoin	selectItem	dispense	refund	restock
IDLE	✔ → HAS_MONEY	✗ insert money	✗ nothing selected	✗ no money	✔ stay
HAS_MONEY	✔ stay (add more)	✔ → DISPENSING	✗ select first	✔ → IDLE	✗ customer buying
DISPENSING	✗ please wait	✗ already dispensing	✔ → IDLE or SOLD_OUT	✗ too late	✗ busy
SOLD_OUT	✗ sold out	✗ sold out	✗ sold out	✔ stay (stranded coin)	✔ → IDLE

Every ✔ becomes a method body with a `setState`. Every ✗ becomes a `throw` with that exact message. **One row = one class. One cell = one method.**

A state that transitions

```
final class IdleState implements VendingMachineState {

    static final IdleState INSTANCE = new IdleState();
    private IdleState() { }

    @Override
    public void insertCoin(VendingMachine machine, int amount) {
        if (amount <= 0) throw new IllegalArgumentException("Coin must be positive");
        machine.addBalance(amount);
        machine.setState(HasMoneyState.INSTANCE);    // the STATE decides what comes next
    }

    @Override
    public void selectItem(VendingMachine machine, String code) {
        throw new IllegalStateException("Insert money first");
    }
    // ...dispense, refund likewise throw; restock is allowed and stays in IDLE

    @Override public String name() { return "IDLE"; }
}
```

A state whose next state depends on data

```
final class DispensingState implements VendingMachineState {

    @Override
    public void dispense(VendingMachine machine) {
        machine.completeSale();           // hands out item + change
        // where we go next depends on the machine's data, so ASK it
        machine.setState(machine.isCompletelyEmpty() ? SoldOutState.INSTANCE
            : IdleState.INSTANCE);
    }
    ...
}
```

Note the shape. The state never inspects the machine's internals. It asks a question (`isCompletelyEmpty()`) and acts on the answer. Encapsulation survives: the states are the only things allowed to call the package-private hooks, and even they go through methods, not fields.

Calling it from main

```
VendingMachine machine = new VendingMachine();    // ← IDLE, balance 0
machine.restock("A1", new Item("A1", "Chips", 40), 2);

machine.insertCoin(20);    // IDLE      -> HAS_MONEY
machine.insertCoin(30);    // HAS_MONEY -> HAS_MONEY  (balance 50)
machine.selectItem("A1");  // HAS_MONEY -> DISPENSING
machine.dispense();        // DISPENSING -> IDLE, change 10 returned

machine.refund();          // throws IllegalStateException("No money to refund")

VendingMachine other = new VendingMachine();
// completely independent balance and stock — shares the state OBJECTS, not the DATA
```

The last two lines are the whole mental model. Two machines, four shared stateless state objects, two separate sets of data. If you can explain those two lines, you understand the pattern.

When NOT to use State

Situation	Use instead
2 states, 1-2 operations	a boolean or an enum field + a small <code>if</code>
The "state" has no behaviour attached — nothing acts differently	a plain enum field. State distributes behaviour; if there is none, there is nothing to distribute
States are fixed by law/spec and will never grow	a switch — you pay the cost and get no return
You add <i>operations</i> more often than <i>states</i>	a switch — see the Expression Problem in section 12
Per-state behaviour is small and the states need no fields	an enum with abstract methods — the middle ground

```
enum TicketStatus {
    OPEN      { void assign(SupportTicket t, String a) { ... } },
    ASSIGNED { void assign(SupportTicket t, String a) { ... } };
    abstract void assign(SupportTicket t, String agent);
}
```

The enum version gives per-state behaviour *and* compiler-enforced completeness with no extra files. It stops being a good fit once a state needs its own fields or injected dependencies. **Mentioning it in an interview shows you chose full State rather than defaulted to it.**

Terminal (final) states

Your doubt: "what do you mean by *final* in the table?" **Final = the end of the line.** A ticket in CLOSED can never leave it. No operation moves it anywhere else, so there is **no `setState` anywhere in `ClosedState`** — all four methods throw. That absence *is* what makes it terminal. This is state-machine vocabulary, **not** the Java keyword `final`. (Confusing, because we also write `final class ClosedState`, which means something completely different: the class cannot be extended.) Most state machines have one: an order DELIVERED, a payment REFUNDED, a machine DECOMMISSIONED. Note it is a **business rule, not a technical necessity** — plenty of real helpdesks do allow reopening a closed ticket. If a spec doesn't say, ask.

11. Exercise 22 — Support ticket workflow

The corrected transition table

	assign(agent)	resolve(actor)	close()	reopen()
OPEN	✔ → ASSIGNED	✗ "assign it first"	✗ "resolve it first"	✗ "not resolved"
ASSIGNED	✔ reassign, stay ASSIGNED	✔ → RESOLVED (<i>only the assigned agent</i>)	✗ "resolve it first"	✗ "not resolved"
RESOLVED	✗ "already resolved"	✗ "already resolved"	✔ → CLOSED	✔ → ASSIGNED (same agent)
CLOSED	✗ final	✗ final	✗ "already closed"	✗ final

Your table error: you allowed `close()` from **every** state. The spec permits it only from RESOLVED. A ticket must be resolved before it can be closed — that's the point of having both statuses.

The vocabulary questions you asked

"What does *agent* mean here?" A support staff member — a human who handles the ticket. "ravi", "meera". Nothing to do with software agents.

"✔ → ASSIGNED (same agent) — what do we write exactly?" Nothing about the agent. The field was never cleared, so it is still set. `reopen` is one line: `ticket.setState(AssignedState.INSTANCE);` "Same agent" means **don't touch the agent field**.

"While closing and reopening we don't need the agent, right?" Correct. Signatures: `void close(SupportTicket t)` and `void reopen(SupportTicket t)`. Only `assign` and `resolve` need a person.

Your Part B mistakes

Your design was right — private constructors with shared instances, a custom unchecked exception, the context starting in OPEN. The code had roughly 30 compile errors and two logic bugs.

#	Mistake	Count	Status
1	No <code>public</code> on interface implementations	16	new
2	Calling <code>setAgent</code> / <code>setState</code> / <code>getAgent</code> with no receiver	7	new, conceptual
3	<code>close()</code> and <code>reopen()</code> both call <code>resolve()</code>	2	new — fatal
4	Decided-but-not-coded (<code>AssignedState.assign</code> throws instead of reassigning)	1	recurring, worst category
5	Skipped requirement (agent-ownership check on resolve)	1	recurring
6	Regressed a lesson from the same session (<code>reopen</code> + agent)	1	recurring
7	Wrong return type (<code>void getAgent()</code> returning a <code>String</code>)	1	recurring since Phase 1
8	Interface incomplete vs what the classes implement	2	new
9	Missing semicolon, wrong arity, missing <code>final</code> , <code>5x public class</code> in one file	4+	recurring

#2 — the conceptual one. Inside `OpenState.assign` you wrote `setAgent(agent);`. In English that reads fine — "set the agent". In Java it means "call `OpenState`'s own `setAgent` method", which does not exist.

A state object owns nothing. It has no agent field and no state field — that's precisely why the ticket is passed in as a parameter. The line must be `ticket.setAgent(agent);`

Before you call a method, ask whose method it is. If the answer is "the ticket's", the line starts with `ticket`.

#3 — the one that running the code would have caught in ten seconds.

```
public void close(SupportTicket st) { this.resolve(this); } // ✗ close() RESOLVES
public void reopen(SupportTicket st) { this.resolve(this); } // ✗ reopen() RESOLVES
```

This compiles happily. Nothing ever reaches `close` or `reopen` on any state, so **ClosedState is unreachable** — all four methods you carefully wrote are dead code. The machine has three states in practice.

Write a ten-line main that walks the happy path and prints the state after each call. It would have caught this instantly, and #4 on the second line.

#4 — decided-but-not-coded, again. Your own table says ASSIGNED + assign = ✔ reassign. Your code threw. Reassignment is the single most common real operation on a helpdesk ticket, and you disabled it.

The self-check that fixes this: after coding, point at the exact line that implements each Part A decision. If you can't find the line, it isn't there.

The corrected code

```
interface TicketState {
    void assign(SupportTicket ticket, String agent);
    void resolve(SupportTicket ticket, String actor);
    void close(SupportTicket ticket);
    void reopen(SupportTicket ticket);
    String name(); // ← you used getName() but never declared it
}

final class OpenState implements TicketState {

    static final OpenState INSTANCE = new OpenState(); // ← yours lacked `final`
    private OpenState() { }
```

```

@Override
public void assign(SupportTicket ticket, String agent) {    // ← `public` required
    if (agent == null || agent.isBlank()) throw new IllegalArgumentException("Agent required");
    ticket.setAgent(agent);                                // ← receiver
    ticket.setState(AssignedState.INSTANCE);
}

@Override public void resolve(SupportTicket t, String actor) {
    throw new InvalidStateException("Assign the ticket before resolving it"); }
@Override public void close(SupportTicket t) {
    throw new InvalidStateException("Resolve the ticket before closing it"); }
@Override public void reopen(SupportTicket t) {
    throw new InvalidStateException("An open ticket has nothing to reopen"); }
@Override public String name() { return "OPEN"; }
}

final class AssignedState implements TicketState {

    static final AssignedState INSTANCE = new AssignedState();
    private AssignedState() { }

    @Override
    public void assign(SupportTicket ticket, String agent) {
        if (agent == null || agent.isBlank()) throw new IllegalArgumentException("Agent required");
        ticket.setAgent(agent);    // reassign – NO setState, staying put is a valid transition
    }

    @Override
    public void resolve(SupportTicket ticket, String actor) {
        if (!ticket.getAgent().equals(actor)) {    // ← missing entirely in yours
            throw new InvalidStateException("Only " + ticket.getAgent() + " can resolve this");
        }
        ticket.setState(ResolvedState.INSTANCE);
    }

    @Override public void close(SupportTicket t) {
        throw new InvalidStateException("Resolve the ticket before closing it"); }
    @Override public void reopen(SupportTicket t) {
        throw new InvalidStateException("Ticket is not resolved yet"); }
    @Override public String name() { return "ASSIGNED"; }
}

final class ResolvedState implements TicketState {

    static final ResolvedState INSTANCE = new ResolvedState();
    private ResolvedState() { }

    @Override public void assign(SupportTicket t, String a) {
        throw new InvalidStateException("Ticket is already resolved"); }
    @Override public void resolve(SupportTicket t, String a) {
        throw new InvalidStateException("Ticket is already resolved"); }
    @Override public void close(SupportTicket ticket) { ticket.setState(ClosedState.INSTANCE); }
    @Override public void reopen(SupportTicket ticket) { ticket.setState(AssignedState.INSTANCE); }
    @Override public String name() { return "RESOLVED"; }
}

final class ClosedState implements TicketState {

    static final ClosedState INSTANCE = new ClosedState();
    private ClosedState() { }

    @Override public void assign(SupportTicket t, String a) {
        throw new InvalidStateException("Cannot assign a closed ticket"); }
    @Override public void resolve(SupportTicket t, String a) {
        throw new InvalidStateException("Cannot resolve a closed ticket"); }
    @Override public void close(SupportTicket t) {
        throw new InvalidStateException("Ticket is already closed"); }
    @Override public void reopen(SupportTicket t) {
        throw new InvalidStateException("A closed ticket cannot be reopened"); }
    @Override public String name() { return "CLOSED"; }
    // no setState anywhere – this is what "terminal" means
}

class SupportTicket {

    private final String id;
    private TicketState state = OpenState.INSTANCE;
    private String agent;

    public SupportTicket(String id) { this.id = id; }

    public void assign(String agent) { state.assign(this, agent); }
    public void resolve(String actor) { state.resolve(this, actor); }
    public void close() { state.close(this); }    // ← yours took a SupportTicket
    public void reopen() { state.reopen(this); }    // ← and both called resolve()

    public String getCurrentStatus() { return state.name(); }

    void setState(TicketState state) { this.state = state; }
    void setAgent(String agent) { this.agent = agent; }
    String getAgent() { return agent; }    // ← yours was declared void
}

```

This version compiles and runs; all transitions and all rejections behave per the table, and the agent survives a reopen.

12. Exercise 22, Part C — the pressure questions

Q7 — "a ticket left in RESOLVED for 7 days closes itself" (you scored 6/10)

✓ **What you got right, and it's the main thing.** No change to `ResolvedState`; the trigger comes from outside the state machine. Most people try to put a timer inside the state class, or add `checkExpiry()` to the interface — which would force all four states to implement it for no reason.

Why you're right: a state class *reacts* to operations, it never *initiates* them. Nothing in there wakes up on its own. So an auto-close must come from outside, and must enter through the normal `ticket.close()` door.

✗ **1. "Add days as an attribute"** — this is the *compute derived values, don't store them* rule.

If `days` is stored, something must **increment it**. Every ticket. Every day. That's a nightly job whose entire purpose is `days++` across a million rows — and if it fails one night, every ticket is silently wrong forever.

Store the timestamp, compute the days.

```
class SupportTicket {
    private Instant stateSince = Instant.now();

    void setState(TicketState state) {
        this.state = state;
        this.stateSince = Instant.now();    // ← the whole change: ONE line
    }
}
```

Note where it goes — in `setState`, not in `ResolvedState`. That makes it work for **every** state automatically. When product next asks "escalate anything unassigned for 2 days", you write zero new plumbing. And `stateSince` can never be stale: nothing has to maintain it.

✗ **2. A separate tracker service** — you already have the tracker. It's the ticket.

A `TrackerService` holding `ticket→days` is a **second copy of state that can disagree with the first**. If a ticket is reopened, who tells the tracker? Miss that message and it auto-closes a ticket an agent is working on. **Don't build a service to remember something the object already knows.**

✗ **3. Observer is the wrong pattern here** — and this is the interesting mistake.

Observer needs an **event**: something happened and a publisher announces it. "Seven days have passed" is not an event. **Nothing happened.** That's the entire point of the requirement — the ticket closes *because* nobody did anything. There is no moment when any code related to this ticket is running, so there is no publisher. Your design had a listener waiting for a notification nobody will ever send.

Carry this one: Observer fires when something **happens**. When the rule is "X **hasn't** happened in N days", there is no event to listen for — you need a **scheduler that goes looking**.

The actual answer

```
@Component
class TicketAutoCloseJob {

    private final TicketRepository repo;
    private final Clock clock;
    private final int days;                // @Value("${ticket.autoclose.days:7}")

    @Scheduled(cron = "0 0 2 * * *")        // 2 a.m. daily
    public void closeStaleResolvedTickets() {
        Instant cutoff = clock.instant().minus(days, ChronoUnit.DAYS);

        for (SupportTicket ticket : repo.findResolvedBefore(cutoff)) { // query, don't scan
            try {
                ticket.close();        // ← the PUBLIC API. Not ClosedState.INSTANCE, not setState.
                repo.save(ticket);      // The state machine still rules.
            } catch (InvalidStateException e) {
                // reopened between the query and now – the machine correctly refused. Skip.
            }
        }
    }
}
```

Files changed: `SupportTicket` (one line + one field). **New:** the job, one repository query. **Untouched:** the interface and all four state classes.

Your follow-up doubts about the job

"Does it loop continuously?" No — it **sleeps**. Spring keeps a scheduler with a timer queue. At 2 a.m. it calls your method once; the method runs, returns, and nothing of yours executes until 2 a.m. tomorrow. Zero CPU in between.

"It loops across all the ticket objects?" **No — the loop is in the database.** The tickets are rows, not objects in memory. `findResolvedBefore` becomes `SELECT * FROM ticket WHERE state='RESOLVED' AND state_since < ?`. With an index, the DB touches only the handful of stale rows. Loading a million tickets nightly to discard 99.99% of them is the naive version.

Say it: "Filter in the query, not in the loop. If a loop's first line is an `if` that skips most rows, that `if` belongs in the WHERE clause."

"What exactly changes in SupportTicket?" One field, one line in `setState`, one getter. You do *not* need a `daysSinceChange()` method — the job computes one cutoff instant and lets the query compare.

Three things an interviewer will push on

- 1. Three servers = three runs.** All three instances wake at 2 a.m. and run the same job on the same rows. Same lesson as "a Singleton is one per JVM". Fix: a distributed lock (ShedLock), leader election, or an external scheduler hitting one endpoint. Your try/catch already makes it *safe*, but it's wasteful — and if the job also sent email, the customer gets three.
- 2. A missed run must self-heal.** The query must be `state_since < cutoff` — **older than** seven days, not *exactly* seven. Then tomorrow's run sweeps up yesterday's stragglers. `DATEDIFF(... ) = 7` would strand them forever after one missed night.
- 3. Inject a Clock .** With `Instant.now()` hardcoded, the only way to test the rule is to wait a week. With an injected clock a unit test moves time forward. A genuine senior-level tell.

Q8 — audit trail: old state, new state, who, when (you scored 5/10)

Your answer: add a `previousState` field; in every state method, call `ticket.setPreviousState(this)` alongside `setState(...)`.

Credit: you reasoned that the old state is available at transition time and that the data belongs on the context — both correct. And your `this` is right: inside `OpenState.assign`, `this` is the `OpenState` instance.

✗ **1. You're still editing every transition site.** Five today — and every state anyone adds in future, the author must remember the extra line, in every method, forever. Nothing enforces it. No compile error, no test failure. Someone forgets once and the audit trail **silently omits that transition**. For a legal requirement, a trail with invisible holes is worse than none, because now you trust it. This is your own recurring failure mode — *decided but not coded* — promoted into a design.

✗ **2. The context already knows.** At the moment `setState` is called, `this.state` has **not been reassigned yet**. It still holds the old state. The "previous state" is free.

✗ **3. A `previousState` field is not an audit trail.** A field holds **one** value; the next transition overwrites it. Legal asked for every transition — that's an append-only **list**, in a table.

The answer: the choke point

`setState` is the **single method every transition must pass through** — and not by luck. It's package-private, so nothing outside can reach the field another way. That decision, made for encapsulation, is what makes this possible.

```
record TransitionRecord(String ticketId, String from, String to, String actor, Instant at) { }

interface TicketAuditLog {
    void record(TransitionRecord r);
    List<TransitionRecord> historyOf(String ticketId);
}

// inside SupportTicket:
void setState(TicketState next, String actor) {
    Instant now = clock.instant();
    audit.record(new TransitionRecord(id, state.name(), next.name(), actor, now));
    //                                ↑ STILL the old state on this line
    this.state = next;
    this.stateSince = now;
}
```

Your doubt: "I did not understand a word of this function."

`audit` and `clock` are **injected fields** on `SupportTicket`, handed in by whoever creates the ticket — the Phase 2 rule: depend on an interface, receive it from outside.

`TransitionRecord` is a tiny immutable object holding the five facts Legal asked for; a Java `record` gives you the constructor and getters free. One instance = one row of the audit table.

`Clock` is a real class in `java.time` — an object whose job is to answer "what time is it?". Not `Instant.now()`, because you can't test that; with a `Clock` field, a test passes a fake clock frozen at any date.

The five arguments: `id` (the ticket's own field) · `state.name()` = **the state being left** · `next.name()` = the state being entered · `actor` (the parameter) · `now`.

The clever bit is the order. Swap the two lines and `state.name()` returns the new state too, so the audit reads "RESOLVED → RESOLVED".

Trace

Ticket T-1042 is ASSIGNED to meera. She calls `ticket.resolve("meera")`:

- `SupportTicket.resolve` → `state.resolve(this, "meera")`, and `state` is `AssignedState.INSTANCE`
- `AssignedState.resolve` checks the agent, then calls `ticket.setState(ResolvedState.INSTANCE, "meera")`
- Inside `setState`: `state.name()` = "ASSIGNED", `next.name()` = "RESOLVED" → row written → then the field moves

And `AssignedState.resolve` contains no audit code. Neither does any other state class. They transition; the recording happens underneath them, whether the author thought about it or not.

The rule: when the same bookkeeping must happen at N call sites, don't write it N times — find the one line they all funnel through and put it there. If there isn't one, **create** one.

Two things to raise unprompted

Reassignment isn't captured. `AssignedState.assign` changes the agent but not the state, so it never touches `setState` and produces no row. `setAgent` is a **second choke point for a different fact**. Flagging this shows you understand a choke point covers exactly one kind of change.

Audit is a required step, not a listener. Legal said every transition *must* be recorded. If the write fails and the transition succeeds anyway, the requirement is broken — so it belongs in the same transaction, not in a try/catch'd listener. Same test as the access card.

Getting the actor there: explicit parameter (honest, testable, but ripples through `close()` and `reopen()`), or `SecurityContextHolder` (zero signature changes, what real Spring apps do, invisible dependency). Name both, pick one.

Q9 — adding an ESCALATED state (you scored 6/10)

✓ You got the shape right: new method on the interface, new class, `AssignedState.escalate` transitions. And you spotted something subtle and good — that the resolve check must *differ per state* (assigned agent in ASSIGNED, manager in ESCALATED).

✗ **"Only one more function and one more class."** Adding a method to an interface breaks every class that implements it. The moment `void escalate(...)` appears in `TicketState`, all four existing classes stop compiling.

File	Change
<code>TicketState</code>	+1 declaration
<code>OpenState</code>	+1 method — throws ("assign it first")
<code>AssignedState</code>	+1 method — → ESCALATED
<code>ResolvedState</code>	+1 method — → ESCALATED (you missed this; the spec says reachable from ASSIGNED and RESOLVED)
<code>ClosedState</code>	+1 method — throws
<code>EscalatedState</code>	new file — 5 operations + <code>name()</code>
<code>SupportTicket</code>	+1 delegating line

Seven files, not two.

The Expression Problem — your compiler argument was backwards

You said: "with if/else we'd edit all four functions; with State we don't." True when adding a **state**. False when adding an **operation** — and here you did both.

	Add a new state	Add a new operation
State pattern	one new class, nothing else touched ✓	edit the interface + every class ✗
if/else on enum	edit every method ✗	one new method ✓

They are mirror images. This is a named thing — the **Expression Problem** — and saying its name lands well. Neither is free; you pick based on which axis your system grows along. Support workflows grow *states* far more often than *operations*, which is why State is right here — but say that as a judgement, not as "State is always fewer edits".

The answer I was fishing for. It is **not** about how many edits. It is about **whether you can forget one**.
With `if/else`, adding ESCALATED means hunting every `if (state == ...)` chain — in the ticket, the UI layer, the reporting job, the notification service. Miss one and nothing breaks at compile time. It falls to the `else`, or silently does nothing, and you find out in production three weeks later.
With State, a missing method is a **compile error**. The build stops.

Say it exactly like this: "The State pattern converts a class of runtime bug into a compile error. It makes it *impossible* to add a state and forget to define what an operation means in it. The if/else version relies on the developer remembering; the State version relies on the compiler."

The design problem you opened and didn't close
"ESCALATED must be resolved by a manager" — how does `EscalatedState.resolve` know? `actor` is a bare `String`; a name doesn't carry a role.
(a) Richer actor — an `Actor` record with `name` + `role`; cleanest, ripples through every signature. **(b) Ask a service** — inject a `UserDirectory`; keeps signatures, adds a dependency to classes that had none. **(c) Check at the boundary** — the controller enforces the role; simplest, but the rule now lives outside the state machine where nobody will find it.
Flagging that a state needs data it doesn't have, before writing code, is the habit that stops decided-but-not-coded methods.

Q10 — two agents call `resolve()` at the same instant (you scored 6/10)

Your scenario (one assigns while another resolves) is genuinely safe — `resolve` re-reads `this.state` at call time, so a stale *screen* doesn't matter. But that means you analysed the case that already works.

The one idea

Every operation does two things: **read** the current state, then **write** the new one. Between them there is a gap, and another thread can slip into it.

Ticket is RESOLVED. Manager A closes it. Meera reopens it. Same moment.

A: reads state → RESOLVED. "Closing is allowed."
B: reads state → RESOLVED. "Reopening is allowed."
A: writes state = CLOSED
B: writes state = ASSIGNED

Final state: ASSIGNED.

A closed ticket just became assigned. Your `ClosedState` forbids that — every method throws. But it never got a say, because B decided *before* A closed it and acted *after*.

Everything the state machine guarantees — including "CLOSED is terminal" — is only true if each transition is atomic. The rules aren't wrong; the window between reading and writing is.

A milder version of the same bug: two managers both close a RESOLVED ticket. Final state is CLOSED (correct), but **two audit rows** say the ticket went RESOLVED → CLOSED, closed by two different people. Legal's record is corrupt.

Separately there is **visibility**: even with no interleaving, thread B might never see A's write — stale CPU cache, the `volatile` lesson. Two problems, not one. `synchronized` fixes both; `volatile` alone fixes only visibility.

Fix for one server

```
public synchronized void close(String actor) { state.close(this, actor); }
public synchronized void reopen(String actor) { state.reopen(this, actor); }
public synchronized String getCurrentStatus() { return state.name(); } // readers too
```

B cannot start until A has finished reading **and** writing. B then reads CLOSED and `ClosedState.reopen` correctly throws.

Your doubt: "ConcurrentHashMap lets two threads into the same function — why doesn't `synchronized`?"

ConcurrentHashMap is not lock-free. It locks *less*. It locks **one bucket** at a time, not the whole map. Two threads on different keys never contend, so they run genuinely in parallel. Two threads on the *same* key — same bucket — and one still waits.

Why can it do that? Because the keys are independent. `put("ravi",x)` and `put("meera",y)` don't affect each other's outcome.

A `SupportTicket` has ONE state field. Every operation reads and writes that one field. There is no independence to exploit, no second bucket to split off.

Coarse locking isn't a limitation of `synchronized` — it's the nature of the data.

Analogy: a hotel locks each room separately so a hundred guests move at once. A single room can't be subdivided — two people can't both renovate it.

Second point people miss: `ConcurrentHashMap` makes each *individual operation* atomic, not your *sequence* of them. That's why `putIfAbsent`, `computeIfAbsent` and `merge` exist — they fold check-then-act into one step. `resolve()` spans several method calls, so no collection could bundle it.

The twist that connects both lessons. `synchronized` on an **instance** method locks *that instance*. Ticket T-1 and ticket T-42 are different objects with different locks — two agents on two tickets never block each other. That is the same idea as per-bucket locking, at object granularity. A static `synchronized` method, or one lock around a whole `TicketService`, would be the disaster.

Lock the smallest thing that must change together. For a map, a bucket. For a ticket, the ticket.

Is `synchronized` enough? Three caveats

- Readers need it too**, or they see stale values.
- You're doing I/O inside the lock** — `audit.record(...)` is a network round-trip while every other thread on that ticket waits. Correct but slow; name it before the interviewer does.
- The one that kills it:** in a real Spring app each request **loads its own `SupportTicket` object** from the database. Two concurrent requests for T-1042 get *two different Java objects* for the same row — two different locks, so nothing is excluded. Last write wins, the other is silently lost.

`synchronized` protects a shared in-memory object. Once the ticket lives in a database, it protects nothing.

Database versioning (optimistic locking) — you asked how it works

Locks are per-JVM. Three servers = three unrelated lock tables = zero mutual exclusion. The guarantee must move to the one thing all three share: **the database**.

The extra column

id	state	agent	version
42	RESOLVED	meera	7

It has no business meaning. It only counts how many times the row has changed.

The rule

Every write says: *"update this row, but only if the version is still what I saw when I read it."*

```
UPDATE ticket
SET state = 'CLOSED', version = 8
WHERE id = 42 AND version = 7
      ↑ this is the whole trick
```

The database reports **how many rows it changed**:

- **1 row** → nobody beat you. Your write landed.
- **0 rows** → the version wasn't 7 anymore. Someone got there first, and your write **did not happen**.

Why this is safe. A single `UPDATE` statement is **atomic** — the database locks that row for the duration of that one statement, so two updates cannot interleave halfway. Exactly one of them finds `version = 7`. That is the part `synchronized` could not give you across servers: the database is the one thing all servers share, so it becomes the referee.

The trace

```
Server A and Server B both read ticket 42. Both see version = 7.

A: UPDATE ... SET state='CLOSED', version=8 WHERE id=42 AND version=7
   → 1 row changed ✓ (row is now CLOSED, version 8)

B: UPDATE ... SET state='ASSIGNED', version=8 WHERE id=42 AND version=7
   → 0 rows changed ✗ (version is 8 now, the WHERE matches nothing)
```

B's reopen is rejected; the ticket stays CLOSED. Without the version column, B's update would simply have overwritten A's — a **lost update**.

In Spring it's one line

```
@Entity
class SupportTicket {
    @Version
    private long version;
}
```

JPA adds `AND version = ?` to every update, bumps the number, and throws `OptimisticLockException` when 0 rows change. You never write the SQL.

What you do when it fails

- **Retry** — reload and re-run against the fresh state. Good for background jobs.
- **Tell the user** — "this ticket was changed by someone else, reload and try again." Good for a UI; a human should see what changed before acting again.

Why "optimistic"? You assume conflicts are rare, so you lock nothing up front and just check at the end whether anyone interfered. Cheap when nothing collides — which is almost always. The opposite is **pessimistic** locking (`SELECT ... FOR UPDATE`), which locks the row the moment you read it. Safer, but everyone waits. Use it only when collisions are common.

Q10 in one breath: "Reading the state and writing it are two steps, so two threads can both pass the check before either writes. Within one JVM I'd make the ticket's methods `synchronized` — and because each ticket is its own lock, different tickets don't contend. But once tickets are loaded per-request from a database, or there are several servers, a Java lock guarantees nothing, so I'd use a version column and let the database reject the second write."

Q11 — "this is over-engineered, justify it" (you scored 7/10 — your best)

✓ Your content was right: 16 branches, editing four switches when a state is added, behaviour scattered by operation instead of grouped by state, SRP and OCP.

✗ **1. "Prone to mistake" is an opinion.** The interviewer can answer "then be careful." Make it concrete — say **which** mistake and **who catches it**: **switch**: forget a case → falls through `default` → ships → wrong behaviour weeks later. **State**: forget a method → **the build fails**. Same mistake; one caught by a human, one by the compiler. That's not "cleaner code", that's a category of bug deleted. Lead with it.

✗ **2. You gave no ground, and the question asked for it.** This matters more than the technical content. "This is over-engineered" is usually a test of whether you can be argued with. Answering only with reasons you're right reads as junior.

When the interviewer is right

- **2 states, 1-2 operations.** Four small branches in one readable method beats an interface plus two classes. State earns its keep around 4 states × 4 operations = 16 branches, so here it's earned. **Say where the line is.**
- **The states never grow.** State pays off when you add states. A workflow fixed by regulation pays the cost and gets no return.
- **You add operations more often than states.** Then State is actively worse — the Expression Problem.

The script "At two states I'd agree — a switch is less code and easier to read. Here it's four states and four operations, so it's sixteen branches spread across four methods, and adding ESCALATED means finding and editing every one. With State, a new state is a new class and the compiler refuses to build until I've said what every operation means in it. That's the trade: more files, but forgetting a case becomes a compile error instead of a production bug. If the workflow were fixed and never growing, I'd take the switch."

Mentioning the **enum-with-abstract-methods** middle ground (section 10) shows you chose full State rather than defaulted to it.

13. Parking Lot — the design round

Phoenix Mall, Pune. A page of requirements from non-technical stakeholders, no pattern names anywhere. Some requirements deliberately need no pattern. This is what a 45-minute interview round actually looks like.

The ten requirements, compressed

R	What they said
R1	Floors with SMALL/MEDIUM/LARGE spots. MOTORCYCLE/CAR/TRUCK. Bike fits anywhere; car fits MEDIUM or LARGE; truck LARGE only.
R2	Pick a spot on arrival. Nearest-to-gate now; floor-by-floor at Diwali; evenly-spread at the new mall. <i>"We change our minds twice a year."</i>
R3	Ticket: id, vehicle number, spot id, entry time. <i>"Those four fields for ten years and I can't imagine it changing."</i>
R4	₹20/40/60 per hour by type, rounded up. First 15 min free. Weekends 1.5×. <i>"New vehicle category about once a year."</i>
R5	4 entry + 4 exit gates at once. <i>"Two cars were sent to B-204. The drivers had a shouting match. This cannot happen again."</i>
R6	LED board, mobile app feed, analytics event at 90% occupancy. <i>"When the board dies, cars still need to park."</i>
R7	Barrier must rise when a ticket is issued. <i>"Don't tell them they're parked when they physically can't get in."</i>
R8	Cash / card / UPI. Cash = change + drawer. Card = gateway + retry. UPI = QR + polling.
R9	FREE / OCCUPIED / OUT_OF_SERVICE. <i>"That's all there's ever been. Nothing else planned."</i>
R10	<i>"Reserve a spot in advance? Maybe next year. We haven't decided."</i>

Q1 — your answers, scored

Req	Your answer	Verdict
R1	Two enums	partial — enums right, you skipped the fit rule
R2	Strategy + Factory	✓
R3	Plain class, 4 fields, constructor	✓ avoided the Builder trap
R4	Strategy + Factory	partial — right patterns, wrong placement of two rules
R5	synchronized on ParkingSpot	✗ locked the wrong thing
R6	Tracker with count maps + Bill Pugh	✗ missed Observer entirely
R7	No pattern, call the barrier directly	✓ excellent
R8	Strategy + Factory	✓
R9	Enum attribute, no pattern	✓
R10	"No pattern, it's doable"	partial — right conclusion, wrong reason

Four clean, three partial, two wrong — **roughly 6.5/10**. R7 and R9 landing correctly is real progress: those are the two judgement calls that tripped you in earlier exercises.

Correction 1 — R1: the enums were the easy half

The actual content of R1 is the **fit rule**. Where does "a car fits MEDIUM or LARGE" live? Not in an `if/else` inside the search loop. Put the behaviour on the enum:

```
enum VehicleType {
    MOTORCYCLE { public boolean fitsIn(SpotType s) { return true; } },
    CAR         { public boolean fitsIn(SpotType s) { return s != SpotType.SMALL; } },
    TRUCK       { public boolean fitsIn(SpotType s) { return s == SpotType.LARGE; } };

    public abstract boolean fitsIn(SpotType spot);
}
```

Adding the electric scooter is then one constant with its own rule, **and the compiler makes you define it** — the enum-with-abstract-methods idea from Q11, doing real work.

Correction 2 — R4: right patterns, two rules in the wrong place

You put the 1.5× weekend multiplier inside each of the three pricing strategies. Three strategies × the same rule = written three times. Add the electric scooter Finance mentioned and its author must remember a fourth time. If they forget, scooters are free on weekends and nobody notices. Same for the free 15 minutes.

Both rules apply to **every** vehicle type. That makes them *shared rules*, not per-type formulas:

```
billable duration = total - 15 min      ← applies to all, computed once
base              = strategy.baseCharge(d) ← the ONLY per-type part
final             = base × (weekend ? 1.5 : 1) ← applies to all, applied once
```

```
interface PricingStrategy {
    long ratePerHour();
    default long baseCharge(Duration billable) {
        long hours = (billable.toMinutes() + 59) / 60;    // rounding lives ONCE
        return hours * ratePerHour();
    }
}

class MotorcyclePricing implements PricingStrategy { public long ratePerHour() { return 20; } }
class CarPricing implements PricingStrategy { public long ratePerHour() { return 40; } }
class TruckPricing implements PricingStrategy { public long ratePerHour() { return 60; } }

class PricingService {
    private static final Duration FREE_WINDOW = Duration.ofMinutes(15);
    private static final double WEEKEND_MULTIPLIER = 1.5;
    private final PricingFactory factory;           // injected

    public long calculateFare(Ticket ticket, LocalDateTime exitTime) {
        Duration total = Duration.between(ticket.entryTime(), exitTime);
        if (total.compareTo(FREE_WINDOW) <= 0) return 0;           // shared rule 1
        Duration billable = total.minus(FREE_WINDOW);

        long base = factory.forVehicle(ticket.vehicle().type()).baseCharge(billable);

        return isWeekend(ticket.entryTime())                     // shared rule 2
            ? Math.round(base * WEEKEND_MULTIPLIER) : base;
    }
}
```

Your doubt: "why does PricingService have its own priceFor? Doesn't it just call the factory?"

It does call the factory — the confusion was that I gave both methods the same name. Renamed, it's obvious:

	priceFor(Duration billable)	calculateFare(Ticket ticket, LocalDateTime exitTime)
Lives on	the strategy	the service
Knows	one type's rate + rounding	the free window, the weekend rule
Varies by vehicle?	yes	no

Worked example — a car, 2h 10m, Saturday: total 130 min → billable 115 min → `CarPricing.baseCharge(115)` = `ceil(115/60)=2 × 40 = 80` → Saturday → `80 × 1.5 = 120`.
Why split at all? All in the service → you'd need `if (type == CAR)`, which is a switch and no longer a Strategy. All in the strategy → the two shared rules get copied three times. The split sits exactly on the line.

Your doubt: "should calculateFare also take a strategy name?"

Inject the factory — yes. Constructor injection, `final` field. **Take a strategy name — no.** The pricing strategy isn't a choice; it's **determined by the data** (`ticket.vehicle().type()`). If you accepted a name, someone could pass "MOTORCYCLE" for a truck.

	Pricing	Spot selection
Who decides?	the data — vehicle type	a human — ops policy
Changes	never per car	twice a year, by decision
So	look it up from the ticket	configure it

If the data determines the strategy, look it up. If a person decides, pass it in.

Two things to ask the stakeholder about R4

Which day decides "weekend"? Entry or exit? A car entering 11pm Sunday and leaving 2am Monday is real.
Is the free 15 minutes a threshold or a deduction? At 75 minutes that's 2 hours vs 1 hour — real money.

Correction 3 — R5: you locked the wrong thing

You said: put the status on `ParkingSpot`, make its methods `synchronized`. That makes the *write* safe. The bug isn't in the write.

```
Gate A: strategy searches the list → B-204 is FREE, picks it
Gate B: strategy searches the list → B-204 is FREE, picks it      ← both decided already
Gate A: b204.occupy() → status = OCCUPIED      ✓ succeeds
Gate B: b204.occupy() → status = OCCUPIED      ✓ ALSO succeeds
```

Both calls succeed. Neither throws — setting an already-OCCUPIED field to OCCUPIED is not an error. **Two drivers are sent to B-204.** You made `occupy()` thread-safe and reproduced the exact complaint in the lobby. The damage was done **during the search**, before either lock was taken. This is the same check-then-act gap as Q10 — which you had just solved correctly for the support ticket.

The fix: search + claim must be one operation

```
class SpotRegistry {

    private final List<ParkingSpot> allSpots;

    public ParkingSpot findAndOccupy(VehicleType vehicle, Gate gate, SpotSelectionStrategy s) {
        ParkingSpot spot;
        synchronized (this) {
            spot = s.select(allSpots, vehicle, gate);    // block, not a method
            if (spot == null) throw new LotFullException(vehicle);    // search
            spot.occupy();    // claim
        }
        publish(new SpotOccupiedEvent(...));    // OUTSIDE the lock
        return spot;
    }

    public synchronized void release(ParkingSpot spot) { spot.release(); }

}
```

Three things to say about this

- 1. `release()` must take the same lock.** A lock only protects data if every path that touches it takes it. Free a spot unlocked and a searching thread sees a half-updated list.
 - 2. Why one lock covers all gates:** `synchronized` locks this — the registry object. All eight gates hold a reference to the **same** registry, so they queue on the same lock. If each gate built its own registry there'd be eight locks and zero protection. Interviewers ask "*what exactly is being locked?*"
 - 3. Keep the critical section small.** Only find-and-claim goes inside. The barrier and the LED notification involve network calls — you don't want every gate in the mall queued behind a slow HTTP request. That's why it's a `synchronized` block, not a `synchronized` method.
- The limit, which you should volunteer:** this works for one JVM. Two servers = two registries = two unrelated locks. Then the guarantee moves to the database: `UPDATE spot SET status='OCCUPIED' WHERE id=? AND status='FREE'`, and zero rows changed means you lost.

The lock-free alternative, if you want to go further

```
class ParkingSpot {
    private final AtomicReference<SpotStatus> status = new AtomicReference<>(SpotStatus.FREE);

    boolean tryOccupy() {
        return status.compareAndSet(SpotStatus.FREE, SpotStatus.OCCUPIED); // check AND set, atomic
    }
}

// registry, no global lock:
for (int attempt = 0; attempt < MAX; attempt++) {
    ParkingSpot candidate = strategy.select(allSpots, vehicle, gate);
    if (candidate == null) throw new LotFullException(vehicle);
    if (candidate.tryOccupy()) return candidate;    // won the race
    // lost – someone claimed it in between. Loop and pick another.
}
```

You can still lose, but you **detect** it and retry, so no spot is handed to two drivers. Same shape as the database `version` column: *change it only if it's still what I saw*. Optimistic locking in memory.

The rule: locking the object you're about to change doesn't help if the **decision** to change it was made earlier. Find the full read-decide-write sequence and make *that* atomic — either by locking around all three steps, or with a compare-and-set that fuses the check and the write.

In an interview: give the `synchronized findAndOccupy` answer first because it's obviously correct, then mention CAS as what you'd do if gate throughput mattered.

Correction 4 — R6: you built the data store and forgot the delivery

✗ **You missed Observer.** R6 names three consumers — LED board, mobile feed, 90%-occupancy analytics. Three unrelated things that care about one event. Your tracker has no way to tell any of them. How does the board find out? Polling?

The giveaway sentence: "*when the board dies, cars still need to park*" is the explicit statement that these are **optional side effects whose failure must not block parking**. That is the definition of a listener. (R7 got the opposite sentence, which is why it's a direct call.)

```
record SpotOccupiedEvent(String spotId, int floor, SpotType type, int freeSpotsNow) { }

interface OccupancyListener {
    void onSpotOccupied(SpotOccupiedEvent event);
    void onSpotReleased(SpotReleasedEvent event);
}

private final List<OccupancyListener> listeners = new CopyOnWriteArrayList<>();

private void publish(SpotOccupiedEvent event) {
    for (OccupancyListener l : listeners) {
        try {
            l.onSpotOccupied(event);
        } catch (RuntimeException e) {
            log.error("Listener {} failed", l.getClass().getSimpleName(), e);
            // swallowed ON PURPOSE — a dead LED board must not stop a car parking
        }
    }
}
```

✗ **You stored derived data.** Two maps of counts is a second copy of the truth. The real truth is the spots. If one count update is ever missed, the board says 12 free while 9 are free, and nobody notices for a month — the same mistake as storing `days` in Q7.

Derive it: `allSpots.stream().filter(ParkingSpot::isFree).count()`. Scanning two thousand spots takes microseconds. Cache only if profiling proves you need to, and then update the cache **inside the same locked block** as the status change.

The 90% listener needs one extra thing. "Fires when occupancy **crosses** 90%" is not "fires when occupancy is above 90%" — above 90%, every event would fire it. It needs the previous value to detect the edge:

```
class OccupancyAnalyticsListener implements OccupancyListener {
    private boolean wasAbove90; // ← STATEFUL, unlike the other two
    public void onSpotOccupied(SpotOccupiedEvent e) {
        boolean isAbove = occupancyPercent(e) >= 90;
        if (isAbove && !wasAbove90) analytics.send("occupancy_crossed_90");
        wasAbove90 = isAbove;
    }
}
```

That field means this listener cannot be shared across servers without moving it somewhere shared.

On Bill Pugh — you keep reaching for it. You need **one instance**, not a **globally reachable** one. Create it at startup and pass it to the gates through their constructors; in Spring that's just `@Service`. One instance, no global access point, trivially testable. **Singleton is a last resort — it's a global variable with better PR.**

Correction 5 — R10: right conclusion, wrong reason

Build nothing. Say so out loud.

"It's doable anyway" is the wrong framing, twice over.

(a) It's not as easy as you said. A reservation is a spot that is neither FREE nor OCCUPIED — *held* for a person, for a window, auto-expiring if they don't show. That's a fourth status with a timeout, which reopens R9. Telling a stakeholder "that's easy" is a commitment you'd regret.

(b) "Doable" isn't the question. Everything is doable. The question is *should you build it now*, and the answer is no: no ticket, no date, no decision.

YAGNI. The cost of guessing wrong is an abstraction shaped for an imagined feature the real one won't fit — and abstractions are much harder to remove than to add.

The script "Nothing for now. There's no spec and no date, so anything I build is a guess. The design is already open where it needs to be — a new selection strategy or pricing rule is one class. When reservations are specced, we'll know whether a spot needs a held-state with expiry, and we'll build the right thing then rather than the thing I imagined today."

A clean design is what makes future features cheap — not speculative hooks.

Q2 — shared vs per-request

Your labels were right. One refinement: there are **three** buckets, and the middle one is easy to miss.

Bucket	Examples	Created
One shared instance	registry, all strategies, all factories, pricing/entry/exit services, listeners	once, at startup
Many, long-lived	ParkingSpot, Gate	at startup, one per physical thing, lives forever
Per request, short-lived	Ticket, Vehicle, events, Receipt	on each vehicle, discarded after

Your "new instance" label covered buckets 2 and 3, which behave very differently: a `ParkingSpot` is created once and mutated for years; a `Ticket` is created fresh each time and never changes.

Why the services are shared: they are **stateless** — the booking arrives as a parameter. **Why the registry must be shared:** it is the single source of truth for occupancy; two registries means the R5 bug returns across servers.

Q4 — why R9 is not the State pattern ✓

Count what State would actually distribute.

	Support ticket	Parking spot
States	4	3
Operations with per-state behaviour	4	0
Behaviours to distribute	16	none

A spot doesn't *do* anything. There is no `spot.park()` that behaves differently when FREE versus OCCUPIED. Nothing asks the spot to make a decision — the registry just reads the status while searching. **The status is data, not behaviour.**

The rule: the State pattern distributes **behaviour** across states. If no behaviour varies by state, there is nothing to distribute — you have a field with three possible values. That's an enum. Three classes with no methods would be pure ceremony.

The sentence that gives it away: *"That's all there's ever been. Nothing else planned."* State pays for itself when states keep being added. **When it would flip:** if reservations arrive, RESERVED has real behaviour — who may claim it, what happens on timeout, whether maintenance can take it. Say that out loud: *"Enum today. If reservations land, the status grows behaviour and I'd revisit it."* It shows you chose rather than defaulted.

Q5 — why R6 and R7 are handled differently (8/10 — your cleanest answer)

✓ You named both halves unprompted: failure impact, and the silent-swallow problem.

Name the test so it's reusable
If this step fails, is the operation still valid?
Yes → listener. **No** → required step, called directly.
LED board dies → the car is still parked in a real spot. **Valid** → listener.
Barrier doesn't open → the car is outside, blocking the entrance, holding a ticket saying it's parked. **Not valid** → direct call.
The same test told you the access card wasn't a listener (co-working exercise) and that a legal audit wasn't a listener (Q8). Three problems, one question.

What you missed: order matters too, not just the mechanism.

1. find and occupy a spot

← required

2. open the barrier

← required; if it throws, RELEASE the spot

3. create the ticket

← required

4. publish the event

← announcements, LAST

If you published first, the LED board would already have decremented the count and analytics already fired — then the barrier fails and you must undo all of it. **Required steps first, announcements last.** Each step runs only if every earlier one succeeded. Same shape as `confirm()` in the co-working exercise.

Your transaction point is half right — the wrong half is interesting. A database transaction can only roll back **database rows**. It cannot roll back a barrier arm that has physically risen, an email already sent, or a payment already captured.
So if the barrier opens and something *after* it fails, `@Transactional` undoes the ticket row — and a car drives in with no ticket recorded. Worse than before. For anything outside the database you need **compensation** — an explicit undo you write yourself:

```
ParkingSpot spot = registry.findAndOccupy(...);
try {
    barrier.open(gate);
} catch (BarrierException e) {
    registry.release(spot);    // ← compensation: give the spot back
    throw e;
}
```

Also: calling an external service *inside* a DB transaction is an anti-pattern on its own — you hold a connection open for a network call.

Two more reasons the barrier can't be a listener, mechanical rather than philosophical: listeners have **no guaranteed order** (the board might update before the car can enter), and listeners are often **async** (with `@Async` the barrier opens on another thread, a moment later — the driver is at the gate *now*).

Q6 — the entry method signature (7/10)

```
public Ticket bookParkingSpot(VehicleType type, Gate gate)    // yours
```

✓ Returns a `Ticket`, takes the `Gate` (the nearest-to-gate strategy needs it), takes the vehicle type (fit rules + pricing).

✗ **Where does the vehicle number come from?** Your own `Ticket` has four fields — `id`, **`vehicleNumber`**, `spot`, `entry time`. This signature cannot supply it. That's decided-but-not-coded showing up at the *signature* level. Good news: catching it here costs nothing; catching it after 200 lines costs an hour.

The fix is nicer than a second parameter: record `Vehicle(String number, VehicleType type) { }` — one parameter carries both facts and they can't get out of sync.

✗ **You didn't answer "what does it throw".** Two things can fail: `LotFullException` and `BarrierFailureException`.

Worth saying out loud: **"lot full" is arguably not exceptional** — it's a normal Tuesday evening. The alternative is `Optional<Ticket>`. At an entry gate the operation genuinely failed and the gate must display something, so an exception reads fine — but **name the choice**, an interviewer will probe it.

✗ **The name undersells it.** `bookParkingSpot` describes step one of four. The method also opens a barrier and issues a ticket. And you didn't say which class it lives in — it belongs in `EntryService`, not the registry.

```
public class EntryService {
    /**
     * @throws LotFullException      no spot fits this vehicle
     * @throws BarrierFailureException barrier didn't open; the spot is released
     */
    public Ticket admitVehicle(Vehicle vehicle, Gate gate) { ... }
}
```

The corrected blueprint

Signatures only. This is what your diagram should look like after all five corrections.

1 • Enums — shared by nature

```
enum SpotType { SMALL, MEDIUM, LARGE }
enum SpotStatus { FREE, OCCUPIED, OUT_OF_SERVICE }
enum PaymentMethod { CASH, CARD, UPI }

enum VehicleType {
    MOTORCYCLE { public boolean fitsIn(SpotType s) { ... } },
    CAR { public boolean fitsIn(SpotType s) { ... } },
    TRUCK { public boolean fitsIn(SpotType s) { ... } };
    public abstract boolean fitsIn(SpotType spot);
}
```

2 • Value objects — new instance per vehicle

```
record Vehicle(String number, VehicleType type) { }
record Ticket(String id, Vehicle vehicle, String spotId, LocalDateTime entryTime) { }
record Receipt(String ticketId, Duration duration, long amount, PaymentMethod method) { }
record SpotOccupiedEvent(String spotId, int floor, SpotType type, int freeSpotsNow) { }
record SpotReleasedEvent(String spotId, int floor, SpotType type, int freeSpotsNow) { }
```

3 • Domain inventory — many instances, created at startup

```
class ParkingSpot {
    private final String id;
    private final int floorNumber;
    private final SpotType type;
    private SpotStatus status;

    boolean isFree();
    void occupy(); void release(); void markOutOfService(); // package-private
    String getId(); int getFloorNumber(); SpotType getType();
}

class Gate {
    private final String id;
    private final int floorNumber; // the strategy needs to know WHERE the gate is
    private final boolean exitGate;
}
```

4 • Spot selection — Strategy + Factory, shared

```
interface SpotSelectionStrategy {
    ParkingSpot select(List<ParkingSpot> spots, VehicleType vehicle, Gate gate);
}
class NearestToGateStrategy implements SpotSelectionStrategy { }
class FloorByFloorStrategy implements SpotSelectionStrategy { }
class EvenlySpreadStrategy implements SpotSelectionStrategy { }

class SpotSelectionFactory { SpotSelectionStrategy forName(String name); }
```

5 • Pricing — Strategy + Factory + Service, shared

(full code in Correction 2)

6 • Payment — Strategy + Factory, shared

```
interface PaymentStrategy { void pay(long amount); } // throws PaymentFailedException
class CashPayment implements PaymentStrategy { } // change + drawer
class CardPayment implements PaymentStrategy { } // gateway + retry
class UpiPayment implements PaymentStrategy { } // QR + polling
class PaymentFactory { PaymentStrategy forMethod(PaymentMethod method); }
```

Note these aren't pure "compute" strategies — they do I/O and can fail, so they throw rather than return void silently.

7 • Observer — shared

```
interface OccupancyListener {
    void onSpotOccupied(SpotOccupiedEvent event);
    void onSpotReleased(SpotReleasedEvent event);
}
class LedBoardListener implements OccupancyListener { }
class MobileFeedListener implements OccupancyListener { }
class OccupancyAnalyticsListener implements OccupancyListener { private boolean wasAbove90; }
```

8 • The registry — one shared instance; the concurrency core

```
class SpotRegistry {
    private final List<ParkingSpot> allSpots;
    private final List<OccupancyListener> listeners = new CopyOnWriteArrayList<>();

    ParkingSpot findAndOccupy(VehicleType v, Gate g, SpotSelectionStrategy s);
    //   synchronized BLOCK around select + occupy only; publishes AFTER the lock
    //   throws LotFullException
    void release(ParkingSpot spot);           // same lock, then publish

    int freeCount();                          // DERIVED – do not store a count
    Map<Integer, Integer> freeCountByFloor();
    Map<SpotType, Integer> freeCountByType();

    void subscribe(OccupancyListener listener);
    private void publish(...);               // try/catch per listener
}
```

9 • External systems — interfaces, injected, shared

```
interface BarrierController { void open(Gate gate); }    // throws BarrierFailureException
interface SpotRepository { List<ParkingSpot> loadAll(); void save(ParkingSpot spot); }

class TicketStore {
    private final Map<String, Ticket> activeTickets = new ConcurrentHashMap<>();
    void put(Ticket t); Ticket get(String ticketId); void remove(String ticketId);
}
```

10 • Orchestration — shared

```
class EntryService {
    private final SpotRegistry registry;
    private final SpotSelectionFactory selectionFactory;
    private final String selectionPolicy;           // from config, e.g. "NEAREST_TO_GATE"
    private final BarrierController barrier;
    private final TicketStore tickets;

    Ticket admitVehicle(Vehicle vehicle, Gate gate);
    // 1. registry.findAndOccupy(...)      required
    // 2. barrier.open(gate)                required – on failure RELEASE the spot, then rethrow
    // 3. build + store the Ticket
}

class ExitService {
    Receipt processExit(String ticketId, Gate gate, PaymentMethod method);
    // 1. look up the ticket      2. compute the fare
    // 3. take payment            - if this fails, DO NOT release the spot
    // 4. release the spot        5. open the barrier      6. remove the ticket
}
```

Note where the selection policy is resolved. EntryService holds the policy name (config) and asks the factory once; the registry just allocates with whatever strategy it's handed. **Keeps business policy out of the concurrency code.**

What changed from your diagram

Your version	Corrected
parkingSpotsPersistence doing 4 jobs (DB access, concurrency, publishing, barrier)	split into SpotRepository · SpotRegistry · EntryService
barrier opened inside the occupy method	moved to EntryService, with spot-release compensation
calculateFare(isWeekend) — no duration	calculateFare(Ticket, exitTime)
getFare()	ratePerHour()
no exit path at all	ExitService, Receipt, TicketStore
bookParkingSpot(VehicleType, Gate)	admitVehicle(Vehicle, Gate)
2 listeners, no interface declared	3 listeners + interface; the analytics one is stateful
isExistGate, 4 gates, gate has only an id	isExitGate, 8 gates, gate knows its floor
counts stored in two maps	counts derived from the spots

Two open questions worth raising yourself

In ExitService, what if payment succeeds and the barrier fails? The customer has paid and can't leave. A DB rollback won't un-charge a card. select() returning null vs Optional. Pick one and be consistent — mixing them is where NPEs come from.

14. Consolidation

Requirement → pattern cheat sheet (all nine patterns so far)

What the requirement sounds like	Answer
"a different formula / rule per type", "we change our minds twice a year"	Strategy
"the type arrives as a String / enum and we must pick the right object"	Factory (usually with Strategy)
"the three systems disagreed in the audit", "one source of truth"	one shared instance (inject it; Singleton only as a last resort)
"4+ fields, several optional, rules spanning two fields, immutable once created"	Builder
"when X happens, also do A, B and C" <i>and</i> "if one fails, X still succeeded"	Observer
"operation is legal in some statuses and illegal in others", 3+ statuses × 3+ operations	State
"when X happens, also do A" <i>and</i> "without A, X isn't really done"	no pattern — a required step, called directly, first
"X hasn't happened in N days"	no pattern — a scheduled job. <i>Not</i> Observer: nothing happened
one rule, unchanged for years; one national rate set by law	no pattern — a method or a constant
3–4 fields, all required, never changes	no pattern — a plain class or a <code>record</code>
a status with no behaviour attached	no pattern — an enum field
"maybe next year, we haven't decided"	nothing . YAGNI

The rules you've accumulated

Area	Rule
Strategy	A strategy decides , it doesn't act .
Factory	A factory returns , it doesn't act .
Strategy vs Factory	Factory answers which class; Builder answers how to assemble one.
Builder	One-field rules in the setter (fail fast); cross-field rules in <code>build()</code> .
Builder	A <code>final</code> collection field is not immutable — <code>List.copyOf(...)</code> .
Validation	Validation rejects bad input; it never silently repairs it.
Observer	If this fails, is the operation still valid? Yes → listener. No → required step.
Observer	Required steps first , announcements last . Listeners have no order and may be async.
Observer	Observer needs an event . Time passing is not an event — use a scheduler.
State	State distributes behaviour . No behaviour per state → it's just an enum field.
State	The context has no if on the state ; public methods are one-line delegation.
State	States are shared stateless singletons ; the context is one per real-world thing .
State	It converts a runtime bug into a compile error . That's the real argument.
Expression Problem	State: new state cheap, new operation expensive. Switch: the reverse. Pick by growth axis.
Choke points	Same bookkeeping at N call sites → find the one line they all pass through. If none, create one.
Derived data	Compute derived values, don't store them . Store the timestamp, not the day count.
Duplication	Rule varies by what the strategy varies by → inside the strategy. Same for all → outside it.
Concurrency	Locking the object you change doesn't help if the decision came earlier. Make read-decide-write atomic.
Concurrency	Lock the smallest thing that must change together — a bucket for a map, the object for an object.
Concurrency	<code>synchronized</code> is one JVM. Across servers, the database is the referee (version column).
Concurrency	Keep the critical section small — no network calls inside the lock.
Failure	A transaction rolls back rows , not barriers, emails or payments. Those need compensation .
Scaling	Stateless objects scale for free; shared state must move to a shared store .
Scaling	Filter in the query , not in the loop.
Design	Never ask the caller for something you can derive from the data.
Design	If the data determines the strategy, look it up . If a person decides, pass it in .
Design	OCP protects you from new behaviour ; it does not protect you from new data .
Design	You need one instance , not a globally reachable one. Inject it.

Updated mistake summary — everything tracked so far

Mistake	First seen	Status
Decided but not coded — Part A says X, the code does the opposite	Phase 3	still recurring (ex 22 Q4)
Skipped requirement — a rule in the spec with no line implementing it	Phase 2	still recurring (agent-ownership check; RESOLVED→ESCALATED)
Regressed a lesson from the same session	ex 21	recurring (reopen + agent)
Missing/wrong return type (<code>void getAgent()</code>)	Phase 1	longest-running
Missing <code>public</code> on interface implementations	ex 22	new, 16 occurrences
Calling a method with no receiver (<code>setAgent(x)</code> vs <code>ticket.setAgent(x)</code>)	ex 22	new, conceptual
Copy-paste bug making code unreachable (<code>close()</code> calling <code>resolve()</code>)	ex 22	new, fatal — running it would catch this
Reading private fields from outside instead of getters	Phase 1	not seen in ex 22 ✓
<code>==</code> on Strings instead of <code>.equals</code>	Phase 1	improving
Interface incomplete vs what the classes implement	ex 22	new
Locking the wrong object	Parking Lot	new
Storing derived data instead of computing it	ex 22 Q7	repeated in Parking Lot R6
Duplicating a shared rule across every strategy	Parking Lot R4	new
Reaching for Bill Pugh when injection would do	Parking Lot R6	recurring habit

The one meta-lesson. Your **design decisions and spoken reasoning are now strong** — Part A scores are consistently 6–8/10 and the judgement calls (listener vs required step, enum vs State, YAGNI) are landing. The persistent gap is **translating decisions into correct code**.

Two habits that close it:

- After coding, point at the exact line that implements each Part A decision.** If you can't find it, it isn't there.
- Write a ten-line `main` and run it** before submitting. It catches unreachable code, wrong transitions, and silent no-ops that no amount of re-reading will.

Pre-submit checklist

- ☐ Every interface method implemented with `public` and `@Override`
- ☐ Every method has a return type, and it matches what the body returns
- ☐ Every call to another object's method starts with that object: `ticket.`, `machine.`, `registry.`
- ☐ One `public class` per file
- ☐ Strings compared with `.equals`; `List` accessed with `.get(i)` not `[i]`
- ☐ Every Part A decision has a line of code I can point at
- ☐ Every requirement in the spec has a line of code I can point at
- ☐ Collections that cross threads are concurrent; listener lists are `CopyOnWriteArrayList`
- ☐ The listener loop has a try/catch per listener
- ☐ Required steps are called directly, before any notification
- ☐ If a required step can fail after an earlier one succeeded, there is compensation
- ☐ Nothing derived is stored
- ☐ I ran a `main` that walks the happy path and prints state at each step

Recall questions — cover the answers

1. What are the three parts of the State pattern, and which one holds the data?
2. Why can state objects be shared singletons while the context cannot?
3. What makes a state "terminal", and how do you see it in the code?
4. Why does <code>reopen()</code> not need to re-set the agent?
5. A state class needs <code>ticket.setAgent(...)</code> , not <code>setAgent(...)</code> . Why?
6. What is the one argument for State that no other approach gives you?
7. What is the Expression Problem, and which way does State lean?
8. Name the middle ground between a switch and full State. When does it stop fitting?
9. "Close any ticket resolved for 7 days" — why is Observer wrong here?
10. Why store <code>stateSince</code> rather than a <code>days</code> counter?
11. Why does putting the audit in <code>setState</code> beat adding a line to every state method?
12. At the moment <code>setState</code> runs, what does <code>this.state</code> hold, and why does that matter?
13. Two threads call <code>close()</code> and <code>reopen()</code> on a RESOLVED ticket. What can go wrong?
14. Why does <code>ConcurrentHashMap</code> allow parallelism that a <code>SupportTicket</code> cannot?
15. Why is <code>synchronized</code> useless once tickets are loaded per-request from a database?
16. Explain optimistic locking in three sentences. What does "0 rows changed" mean?
17. Give the one-line concession when an interviewer calls State over-engineered.
18. Parking lot: why doesn't <code>synchronized occupy()</code> fix the double-allocation bug?
19. Which sentence in R6 tells you those three consumers are listeners?
20. Which sentence in R7 tells you the barrier is <i>not</i> a listener?
21. Why must the barrier open <i>before</i> the event is published?
22. Why is the free-15-minutes rule in the service and the ₹40 rate in the strategy?
23. Why does <code>calculateFare</code> not take a strategy name, while spot selection does?
24. Why is a parking spot's status an enum and a ticket's status a set of classes?
25. What does a database transaction <i>not</i> roll back, and what do you use instead?
26. Why is "it's easily doable" the wrong answer to a speculative feature request?
27. What are the three lifetime buckets of objects in the parking lot design?
28. Why create the registry at startup and inject it rather than making it a Singleton?